EJERCICIO. Practicas con Interfaces

Objetivo: practicar interfaces entendiendo el “por qué” y el “cómo”. Cada solución va comentada para que puedas seguir el razonamiento.

Incluye ejercicios para practicar + solución propuesta.

Consejo: intenta resolver cada ejercicio sin mirar la solución. Luego compara y entiende los comentarios.

Plantilla mental- IMPORTANTE

Cuando trabajas con interfaces, piensa siempre en estas 3 ideas:

- 1) La interfaz define el CONTRATO (qué se puede hacer).
- 2) La clase concreta da la IMPLEMENTACIÓN (cómo se hace).
- 3) El código cliente debería usar la interfaz (para no depender de una clase concreta).

Ejercicio 1. Interfaz mínima: Arrancar

Enunciado:

Crea una interfaz IArrancable con un método void Arrancar(). Implementa una clase Coche que muestre “Coche arrancado”. En Main crea un Coche y llama a Arrancar().

Solución propuesta:

```
using System;

public interface IArrancable
{
    // CONTRATO: cualquier clase que implemente IArrancable
    // DEBE tener un método Arrancar().
    void Arrancar();
}

public class Coche : IArrancable
{
    // IMPLEMENTACIÓN: aquí definimos "cómo" arranca un coche.
    public void Arrancar()
    {
        Console.WriteLine("Coche arrancado");
    }
}

public class Program
```

```
{
    public static void Main()
    {
        // Creamos la clase concreta:
        Coche miCoche = new Coche();

        // Llamamos al método definido por la interfaz.
        // En este ejercicio aún usamos Coche directamente,
        // pero ya estamos cumpliendo el contrato IArrancable.
        miCoche.Arrancar();
    }
}
```

Ejercicio 2. Dos clases con la misma interfaz: Notificar

Enunciado:

Crea INotificable con void Notificar(string msg). Implementa Email y Push. En Main llama a Notificar en ambas.

Solución propuesta (muy comentada):

```
using System;

public interface INotificable
{
    // CONTRATO: “puedo notificar un mensaje”
    void Notificar(string msg);
}

public class Email : INotificable
{
    // IMPLEMENTACIÓN para Email
    public void Notificar(string msg)
    {
        Console.WriteLine($"[EMAIL] {msg}");
    }
}

public class Push : INotificable
{
    // IMPLEMENTACIÓN para Push
    public void Notificar(string msg)
    {
        Console.WriteLine($"[PUSH] {msg}");
    }
```

```
}

public class Program
{
    static void Main()
    {
        // Aquí creamos dos objetos diferentes,
        // pero ambos cumplen el mismo contrato INotificable.
        INotificable n1 = new Email();
        INotificable n2 = new Push();

        // Como trabajamos con la interfaz, la llamada es igual:
        n1.Notificar("Hola desde el sistema");
        n2.Notificar("Hola desde el sistema");
    }
}
```

Ejercicio 3. Lista polimórfica (interfaz + List)

Enunciado:

Crea 3 clases (Email, Sms, WhatsApp) que implementen INotificable. Mételas en una List<INotificable> y recórrela.

Solución propuesta (muy comentada):

```
using System;
using System.Collections.Generic;

public interface INotificable
{
    void Notificar(string msg);
}

public class Email : INotificable
{
    public void Notificar(string msg) => Console.WriteLine($"[EMAIL] {msg}");
}

public class Sms : INotificable
{
    public void Notificar(string msg) => Console.WriteLine($"[SMS] {msg}");
}

public class WhatsApp : INotificable
{
```

```
    public void Notificar(string msg) => Console.WriteLine($"[WHATSAPP] {msg}");
}

public class Program
{
    static void Main()
    {
        // La magia de la interfaz:
        // una sola lista puede guardar objetos de clases distintas
        // si todas implementan INotificable.
        List<INotificable> canales = new List<INotificable>
        {
            new Email(),
            new Sms(),
            new WhatsApp()
        };

        // Polimorfismo: tratamos a todos igual (como INotificable),
        // pero cada uno se comporta diferente al ejecutar Notificar().
        foreach (INotificable canal in canales)
        {
            canal.Notificar("Mensaje de prueba");
        }
    }
}
```

Ejercicio 4. Interfaz con propiedad: IIdentificable

Enunciado:

Crea una interfaz IIdentificable con int Id { get; }. Implementa Persona con Id y Nombre. Crea una Persona y muestra su Id.

Solución propuesta (muy comentada):

```
using System;

public interface IIdentificable
{
    // CONTRATO: el objeto expone un Id de solo lectura (get).
    int Id { get; }
}

public class Persona : IIdentificable
{
    // Cumplimos el contrato: implementamos la propiedad Id.
```

```
    public int Id { get; }

    // Propiedad adicional NO obligatoria por la interfaz.
    public string Nombre { get; set; }

    public Persona(int id, string nombre)
    {
        Id = id;
        Nombre = nombre;
    }
}

public class Program
{
    static void Main()
    {
        Persona p = new Persona(10, "Ana");

        // Aunque p sea Persona, sabemos que "como mínimo" cumple IIdentificable.
        Console.WriteLine($"Id = {p.Id}, Nombre = {p.Nombre}");
    }
}
```

Ejercicio 5. Implementación explícita (dos interfaces con mismo método)

Enunciado:

Crea dos interfaces IA y IB que tengan ambas void Ejecutar(). Implementa una clase X que implemente las dos usando implementación explícita.

Solución propuesta (muy comentada):

```
using System;

public interface IA { void Ejecutar(); }
public interface IB { void Ejecutar(); }

public class X : IA, IB
{
    // IMPLEMENTACIÓN EXPLÍCITA:
    // No se puede llamar con x.Ejecutar() directamente.
    // Hay que convertir a IA o IB.
    void IA.Ejecutar()
    {
        Console.WriteLine("Ejecutar desde IA");
    }
```

```
    void IB.Ejecutar()
    {
        Console.WriteLine("Ejecutar desde IB");
    }
}

public class Program
{
    static void Main()
    {
        X x = new X();

        // Convertimos a la interfaz deseada para elegir la implementación.
        IA a = x;
        IB b = x;

        a.Ejecutar(); // llama a IA.Ejecutar
        b.Ejecutar(); // llama a IB.Ejecutar
    }
}
```

Ejercicio 6. Método que recibe una interfaz (desacoplar)

Enunciado:

Crea IDescontable con decimal AplicarDescuento(decimal precio). Crea un método CalcularPrecioFinal(IDescontable d, decimal precio). Pruébalo con un descuento del 10%.

Solución propuesta (muy comentada):

```
using System;

public interface IDescontable
{
    // CONTRATO: dado un precio, devuelve el precio con descuento aplicado.
    decimal AplicarDescuento(decimal precio);
}

public class Descuento10 : IDescontable
{
    public decimal AplicarDescuento(decimal precio)
    {
        // 10% de descuento => pagas el 90%.
        return precio * 0.90m;
    }
```

```
}

public class Program
{
    // El programa NO depende de "Descuento10" directamente.
    // Solo depende del contrato IDescontable.
    public static decimal CalcularPrecioFinal(IDescontable descuento, decimal
precio)
    {
        return descuento.AplicarDescuento(precio);
    }

    static void Main()
    {
        IDescontable d = new Descuento10();

        decimal precio = 100m;
        decimal final = CalcularPrecioFinal(d, precio);

        Console.WriteLine($"Precio original: {precio}");
        Console.WriteLine($"Precio final: {final}");
    }
}
```

Ejercicio 7. Repositorio en memoria (CRUD simple)

Enunciado:

Crea IProvinciaRepository con GetAll/Add/Update/Delete. Implementa ProvinciaRepositoryMemoria con una lista.

Solución propuesta (muy comentada):

```
using System;
using System.Collections.Generic;
using System.Linq;

public class Provincia
{
    public int Id { get; set; }
    public string Nombre { get; set; } = "";
}

public interface IProvinciaRepository
{
    // CONTRATO: operaciones básicas (CRUD)
```

```
    List<Provincia> GetAll();
    void Add(string nombre);
    bool Update(int id, string nuevoNombre);
    bool Delete(int id);
}

public class ProvinciaRepositoryMemoria : IProvinciaRepository
{
    // “Base de datos” en memoria
    private readonly List<Provincia> _data = new List<Provincia>();

    public List<Provincia> GetAll()
    {
        // Devolvemos una lista ordenada por Id (comodidad para mostrar).
        return _data.OrderBy(p => p.Id).ToList();
    }

    public void Add(string nombre)
    {
        // Generamos un Id nuevo (como haría una IDENTITY en SQL).
        int nuevoId = _data.Count == 0 ? 1 : _data.Max(p => p.Id) + 1;

        _data.Add(new Provincia { Id = nuevoId, Nombre = nombre });
    }

    public bool Update(int id, string nuevoNombre)
    {
        // Buscamos la provincia
        Provincia? p = _data.FirstOrDefault(x => x.Id == id);
        if (p == null) return false;

        // Actualizamos
        p.Nombre = nuevoNombre;
        return true;
    }

    public bool Delete(int id)
    {
        Provincia? p = _data.FirstOrDefault(x => x.Id == id);
        if (p == null) return false;

        _data.Remove(p);
        return true;
    }
}
```

Ejercicio 8. Inyección de dependencia simple (constructor)

Enunciado:

Crea una clase ServicioProvincias que reciba IProvinciaRepository por constructor y tenga un método Listar().

Solución propuesta (muy comentada):

```
using System;

public class ServicioProvincias
{
    // Guardamos el contrato, no una clase concreta.
    private readonly IProvinciaRepository _repo;

    // INYECCIÓN DE DEPENDENCIA:
    // desde fuera me pasan la implementación concreta.
    public ServicioProvincias(IProvinciaRepository repo)
    {
        _repo = repo;
    }

    public void Listar()
    {
        foreach (var p in _repo.GetAll())
        {
            Console.WriteLine($"{p.Id} - {p.Nombre}");
        }
    }
}
```

Ejercicio 9. Cambiar implementación sin romper el programa

Enunciado:

Crea dos repositorios: uno en memoria y otro “fake” que devuelva 2 provincias fijas. Cambia solo la línea de creación.

Solución propuesta (muy comentada):

```
using System;
using System.Collections.Generic;

public class Provincia
{
    public int Id { get; set; }
    public string Nombre { get; set; } = "";
}
```

```
public interface IProvinciaRepository
{
    List<Provincia> GetAll();
    void Add(string nombre);
    bool Update(int id, string nuevoNombre);
    bool Delete(int id);
}

public class ProvinciaRepositoryFake : IProvinciaRepository
{
    // Implementación “de mentira” para pruebas/demos.
    public List<Provincia> GetAll() => new List<Provincia>
    {
        new Provincia { Id = 1, Nombre = "Madrid" },
        new Provincia { Id = 2, Nombre = "Sevilla" }
    };

    public void Add(string nombre) { /* no hace nada */ }
    public bool Update(int id, string nuevoNombre) => false;
    public bool Delete(int id) => false;
}

public class Program
{
    static void Main()
    {
        // Cambias SOLO esta línea:
        IProvinciaRepository repo = new ProvinciaRepositoryFake();

        // El resto del programa NO cambia, porque depende del contrato.
        foreach (var p in repo.GetAll())
            Console.WriteLine($"{p.Id} - {p.Nombre}");
    }
}
```

Ejercicio 10. Sistema de pagos (interfaz como estrategia)

Enunciado:

Crea IPago con bool Pagar(decimal importe). Implementa Tarjeta y Bizum. Haz un método ProcesarPago(IPago metodo, decimal importe).

Solución propuesta (muy comentada):

```
using System;
```

```
public interface IPago
{
    // CONTRATO: “puedo intentar cobrar un importe”
    // Devuelve true si el pago se realizó correctamente.
    bool Pagar(decimal importe);
}

public class Tarjeta : IPago
{
    public bool Pagar(decimal importe)
    {
        // En un sistema real aquí habría conexión a pasarela de pago.
        Console.WriteLine($"Pago con tarjeta: {importe:0.00} €");
        return true;
    }
}

public class Bizum : IPago
{
    public bool Pagar(decimal importe)
    {
        Console.WriteLine($"Pago con Bizum: {importe:0.00} €");
        return true;
    }
}

public class Program
{
    public static void ProcesarPago(IPago metodo, decimal importe)
    {
        // No nos importa si es Tarjeta o Bizum: solo que cumpla IPago.
        bool ok = metodo.Pagar(importe);
        Console.WriteLine(ok ? "Pago OK" : "Pago fallido");
    }

    static void Main()
    {
        ProcesarPago(new Tarjeta(), 19.99m);
        ProcesarPago(new Bizum(), 5.50m);
    }
}
```

Ejercicio 11. Comparador genérico por interfaz

Enunciado:

Crea IComparador<T> con int Comparar(T a, T b). Implementa ComparadorPorLongitud para string y compara dos textos.

Solución propuesta (muy comentada):

```
using System;

public interface IComparador<T>
{
    // Devuelve: -1 si a<b, 0 si iguales, 1 si a>b (según el criterio elegido).
    int Comparar(T a, T b);
}

public class ComparadorPorLongitud : IComparador<string>
{
    public int Comparar(string a, string b)
    {
        // Criterio: longitud del texto
        if (a.Length < b.Length) return -1;
        if (a.Length > b.Length) return 1;
        return 0;
    }
}

public class Program
{
    static void Main()
    {
        // Usamos la interfaz => mañana podrías comparar por orden alfabético
        // cambiando SOLO la clase concreta.
        IComparador<string> comp = new ComparadorPorLongitud();

        string t1 = "Hola";
        string t2 = "Hola mundo";

        int r = comp.Comparar(t1, t2);

        if (r < 0) Console.WriteLine("t1 es más corto que t2");
        else if (r > 0) Console.WriteLine("t1 es más largo que t2");
        else Console.WriteLine("tienen la misma longitud");
    }
}
```

Ejercicio 12. Logger por interfaz

Enunciado:

Crea ILogger con Info/Error. Implementa ConsoleLogger y úsalo para registrar un proceso con try/catch.

Solución propuesta (muy comentada):

```
using System;

public interface ILogger
{
    // CONTRATO: registrar mensajes informativos y de error.
    void Info(string msg);
    void Error(string msg);
}

public class ConsoleLogger : ILogger
{
    public void Info(string msg) => Console.WriteLine($"[INFO] {msg}");
    public void Error(string msg) => Console.WriteLine($"[ERROR] {msg}");
}

public class Program
{
    static void Main()
    {
        ILogger logger = new ConsoleLogger();

        logger.Info("Inicio del programa");

        try
        {
            int a = 10;
            int b = 0;

            // Provocamos un error para ver el logger funcionando
            int c = a / b;

            logger.Info($"Resultado: {c}");
        }
        catch (Exception ex)
        {
            logger.Error($"Ha ocurrido un error: {ex.Message}");
        }

        logger.Info("Fin del programa");
```

```
    }
}
```

Ejercicio 13. Regla de validación por interfaz

Enunciado:

Crea IReglaValidacion con EsValido(texto) y MensajeError. Implementa ReglaMinLongitud(>=5) y pruébala.

Solución propuesta (muy comentada):

```
using System;

public interface IReglaValidacion
{
    bool EsValido(string texto);
    string MensajeError { get; }
}

public class ReglaMinLongitud : IReglaValidacion
{
    private readonly int _min;

    public ReglaMinLongitud(int min) => _min = min;

    public string MensajeError => $"El texto debe tener al menos {_min}
caracteres.";

    public bool EsValido(string texto)
    {
        // Validación simple: no vacío y con longitud mínima
        return !string.IsNullOrWhiteSpace(texto) && texto.Length >= _min;
    }
}

public class Program
{
    static void Main()
    {
        IReglaValidacion regla = new ReglaMinLongitud(5);
        string input = "Hola";

        if (!regla.EsValido(input))
            Console.WriteLine(regla.MensajeError);
        else
            Console.WriteLine("Texto válido");
```

```
    }
}
```

Ejercicio 14. Varias reglas (lista de interfaces)

Enunciado:

Crea dos reglas: mínima longitud (>=5) y que contenga "@". Aplica ambas a un email y muestra errores.

Solución propuesta (muy comentada):

```
using System;
using System.Collections.Generic;

public interface IReglaValidacion
{
    bool EsValido(string texto);
    string MensajeError { get; }
}

public class ReglaMinLongitud : IReglaValidacion
{
    private readonly int _min;
    public ReglaMinLongitud(int min) => _min = min;
    public string MensajeError => $"Debe tener al menos {_min} caracteres.";
    public bool EsValido(string texto) => !string.IsNullOrWhiteSpace(texto) &&
texto.Length >= _min;
}

public class ReglaContieneArroba : IReglaValidacion
{
    public string MensajeError => "Debe contener '@'.";
    public bool EsValido(string texto) => !string.IsNullOrWhiteSpace(texto) &&
texto.Contains("@");
}

public class Program
{
    static void Main()
    {
        // Lista de reglas => cada regla es una clase distinta,
        // pero todas cumplen IReglaValidacion.
        List<IReglaValidacion> reglas = new()
        {
            new ReglaMinLongitud(5),
            new ReglaContieneArroba()
```

```
        };

        string email = "anaejemplo.com"; // mal (sin arroba)

        bool ok = true;
        foreach (var r in reglas)
        {
            if (!r.EsValido(email))
            {
                ok = false;
                Console.WriteLine($"Error: {r.MensajeError}");
            }
        }

        Console.WriteLine(ok ? "Email válido" : "Email NO válido");
    }
}
```

Ejercicio 15. Reloj por interfaz (facilita pruebas)

Enunciado:

Crea IClock con Now. Implementa SystemClock y FakeClock. Haz un SaludoService que salude según la hora.

Solución propuesta (muy comentada):

```
using System;

public interface IClock
{
    DateTime Now { get; }
}

public class SystemClock : IClock
{
    // Implementación real
    public DateTime Now => DateTime.Now;
}

public class FakeClock : IClock
{
    // Implementación de prueba: hora fija
    private readonly DateTime _fixed;
    public FakeClock(DateTime fixedTime) => _fixed = fixedTime;
    public DateTime Now => _fixed;
}
```

```
public class SaludoService
{
    private readonly IClock _clock;

    // Dependemos del contrato => en pruebas inyectamos FakeClock
    public SaludoService(IClock clock) => _clock = clock;

    public string ObtenerSaludo()
    {
        int h = _clock.Now.Hour;
        if (h < 12) return "Buenos días";
        if (h < 20) return "Buenas tardes";
        return "Buenas noches";
    }
}

public class Program
{
    static void Main()
    {
        Console.WriteLine(new SaludoService(new SystemClock()).ObtenerSaludo());

        // Prueba controlada:
        Console.WriteLine(new SaludoService(new FakeClock(new DateTime(2026, 2, 15,
9, 0, 0))).ObtenerSaludo());
    }
}
```

Ejercicio 16. Exportador por interfaz (CSV / Texto)

Enunciado:

Crea IExportador con Exportar(List<Provincia>). Implementa ExportadorCsv y ExportadorTexto. Pruébalos con la misma lista.

Solución propuesta (muy comentada):

```
using System;
using System.Collections.Generic;
using System.Text;

public class Provincia
{
    public int Id { get; set; }
    public string Nombre { get; set; } = "";
```

```
}

public interface IExportador
{
    // CONTRATO: transformar datos a un string de salida.
    string Exportar(List<Provincia> datos);
}

public class ExportadorCsv : IExportador
{
    public string Exportar(List<Provincia> datos)
    {
        // CSV: cabecera + líneas
        StringBuilder sb = new StringBuilder();
        sb.AppendLine("Id;Nombre");

        foreach (var p in datos)
            sb.AppendLine($"{p.Id};{p.Nombre}");

        return sb.ToString();
    }
}

public class ExportadorTexto : IExportador
{
    public string Exportar(List<Provincia> datos)
    {
        StringBuilder sb = new StringBuilder();
        sb.AppendLine("=== PROVINCIAS ===");

        foreach (var p in datos)
            sb.AppendLine($"- {p.Id}: {p.Nombre}");

        return sb.ToString();
    }
}

public class Program
{
    static void Main()
    {
        var datos = new List<Provincia>
        {
            new Provincia { Id = 1, Nombre = "Madrid" },
            new Provincia { Id = 2, Nombre = "Sevilla" }
        };
```

```
        // Misma entrada, diferentes salidas => estrategia por interfaz
        IExportador exp1 = new ExportadorCsv();
        IExportador exp2 = new ExportadorTexto();

        Console.WriteLine(exp1.Exportar(datos));
        Console.WriteLine(exp2.Exportar(datos));
    }
}
```

Ejercicio 17. Proveedor de texto por interfaz

Enunciado:

Crea IProveedorTexto con ObtenerTexto(). Implementa ProveedorConsola (lee consola) y ProveedorFijo (texto fijo).

Solución propuesta (muy comentada):

```
using System;

public interface IProveedorTexto
{
    string ObtenerTexto();
}

public class ProveedorConsola : IProveedorTexto
{
    public string ObtenerTexto()
    {
        Console.Write("Introduce un texto: ");
        return Console.ReadLine() ?? "";
    }
}

public class ProveedorFijo : IProveedorTexto
{
    private readonly string _texto;
    public ProveedorFijo(string texto) => _texto = texto;

    public string ObtenerTexto() => _texto;
}

public class Program
{
    static void Main()
```

```
    {
        // Cambia una sola línea y el programa se mantiene:
        IProveedorTexto proveedor = new ProveedorConsola();
        // IProveedorTexto proveedor = new ProveedorFijo("Texto de prueba");

        string texto = proveedor.ObtenerTexto();
        Console.WriteLine($"Has obtenido: {texto}");
    }
}
```

Ejercicio 18. Cálculo de envío por interfaz

Enunciado:

Crea ICalculoEnvio con Calcular(pesoKg). Implementa EnvioEstandar y EnvioUrgente. Usa ambos con el mismo método MostrarCoste.

Solución propuesta (muy comentada):

```
using System;

public interface ICalculoEnvio
{
    decimal Calcular(decimal pesoKg);
}

public class EnvioEstandar : ICalculoEnvio
{
    public decimal Calcular(decimal pesoKg)
    {
        return 2m + (1m * pesoKg);
    }
}

public class EnvioUrgente : ICalculoEnvio
{
    public decimal Calcular(decimal pesoKg)
    {
        return 5m + (2m * pesoKg);
    }
}

public class Program
{
    public static void MostrarCoste(ICalculoEnvio calc, decimal peso)
    {
```

```
        Console.WriteLine($"Coste: {calc.Calcular(peso):0.00} €");
    }

    static void Main()
    {
        MostrarCoste(new EnvioEstandar(), 3m);
        MostrarCoste(new EnvioUrgente(), 3m);
    }
}
```

Ejercicio 19. Varias capacidades (IGuardar + ICargar)

Enunciado:

Crea IGuardar y ICargar. Implementa ConfigApp que implemente ambas. Demuestra llamadas como clase y como interfaz.

Solución propuesta (muy comentada):

```
using System;

public interface IGuardar
{
    void Guardar();
}

public interface ICargar
{
    void Cargar();
}

public class ConfigApp : IGuardar, ICargar
{
    // Estado de ejemplo
    public string Tema { get; set; } = "Claro";

    public void Guardar()
    {
        Console.WriteLine($"Guardando... Tema={Tema}");
    }

    public void Cargar()
    {
        Console.WriteLine("Cargando...");
        Tema = "Oscuro";
        Console.WriteLine($"Cargado. Tema={Tema}");
```

```
    }
}

public class Program
{
    static void Main()
    {
        ConfigApp config = new ConfigApp();
        config.Guardar();
        config.Cargar();

        // Como interfaces (contratos)
        IGuardar g = config;
        ICargar c = config;

        g.Guardar();
        c.Cargar();
    }
}
```

Ejercicio 20. Mini-proyecto: menú que usa repositorio por interfaz

Enunciado:

Crea un mini menú (Listar/Insertar/Salir) que trabaje con IProvinciaRepository. Usa ProvinciaRepositoryMemoria. Objetivo: que el menú NO dependa de la clase concreta.

Solución propuesta (muy comentada):

```
using System;
using System.Collections.Generic;
using System.Linq;

public class Provincia
{
    public int Id { get; set; }
    public string Nombre { get; set; } = "";
}

public interface IProvinciaRepository
{
    // Contrato mínimo para este mini-proyecto
    List<Provincia> GetAll();
    void Add(string nombre);
}
```

```
public class ProvinciaRepositoryMemoria : IProvinciaRepository
{
    private readonly List<Provincia> _data = new();

    public List<Provincia> GetAll() => _data.OrderBy(p => p.Id).ToList();

    public void Add(string nombre)
    {
        int nuevoId = _data.Count == 0 ? 1 : _data.Max(p => p.Id) + 1;
        _data.Add(new Provincia { Id = nuevoId, Nombre = nombre });
    }
}

public class Program
{
    static void Main()
    {
        // CLAVE: dependemos de la interfaz, no de la implementación.
        IProvinciaRepository repo = new ProvinciaRepositoryMemoria();

        while (true)
        {
            Console.WriteLine("\n1) Listar  2) Insertar  0) Salir");
            Console.Write("Opción: ");
            string op = Console.ReadLine() ?? "";

            if (op == "0") break;

            if (op == "1")
            {
                foreach (var p in repo.GetAll())
                    Console.WriteLine($"{p.Id} - {p.Nombre}");
            }
            else if (op == "2")
            {
                Console.Write("Provincia: ");
                string nombre = (Console.ReadLine() ?? "").Trim();

                if (nombre.Length == 0)
                {
                    Console.WriteLine("Nombre vacío, no se inserta.");
                    continue;
                }

                repo.Add(nombre);
                Console.WriteLine("Insertado.");
```

```
            }
            else
            {
                Console.WriteLine("Opción no válida.");
            }
        }
    }
}
```